STAYKARO PRIVATE LIMITED

AI CALLER AGENT

API Specification

Complete endpoint reference for the platform backend

Version 1.1
All endpoints implemented in services/api/ unless marked internal
Companion to the Developer Handbook and Database Design documents

Table of Contents

1. What Is an API, and Why Do We Version, Paginate, and Rate-Limit It?

CONCEPT · APIs, in the context of this document

WHAT IT IS

An API endpoint is one specific, named door into the backend -- a fixed URL and HTTP method that does exactly one thing, documented precisely enough that a frontend developer never has to guess what it returns.

REAL-WORLD ANALOGY

This document is the restaurant's actual printed menu -- not just "you can order food" (that's the concept, covered in the Developer Handbook §2), but the full list of dishes, their exact ingredients, and what happens if the kitchen is out of something.

WHY STAYKARO USES IT

Nihal's dashboards and Shivashree's voice gateway both talk to Nishkala's backend exclusively through these endpoints. None of the three needs to read another's source code to integrate correctly -- they read this document instead.

TECHNICAL IMPLEMENTATION

Every endpoint below is implemented in `services/api/` and follows the API Design Standards in the Developer Handbook, §6.

1.1 Versioning

Every endpoint is prefixed `/api/v1/`. If a future change would break existing clients (removing a field, changing a type), it ships as `/api/v2/` alongside the old version, rather than silently changing v1's behavior underneath whoever is still calling it.

1.2 Pagination

CONCEPT · Pagination

WHAT IT IS

Pagination means returning a large result set in smaller pages, rather than all at once.

REAL-WORLD ANALOGY

It's the difference between a librarian handing you one book at a time as you ask for it, versus dumping the entire library on your desk because you asked for "a book."

WHY STAYKARO USES IT

A busy tenant might have thousands of calls. Returning all of them on one request would be slow for the backend to build and slow for the dashboard to render.

TECHNICAL IMPLEMENTATION

List endpoints (calls, documents, invoices) accept `?page=1&page_size=25` and return a `pagination` object alongside the results: `{total, page, page_size, has_next}`.

1.3 Rate limiting

CONCEPT · Rate limiting

WHAT IT IS

Rate limiting caps how many requests a client can make in a given time window.

REAL-WORLD ANALOGY

It's a bouncer at a door with a one-in-one-out policy during a fire code crowd limit -- not because everyone outside is unwelcome, but because letting everyone in at once would break something for everyone already inside.

WHY STAYKARO USES IT

It protects the platform from a single misbehaving client (a buggy dashboard polling too aggressively, or a malicious actor) from degrading service for every other tenant on shared infrastructure.

TECHNICAL IMPLEMENTATION

Enforced at two layers per the Architecture Spec §27 and Execution Plan §L: Nginx applies a coarse per-IP limit, and the API applies a finer per-tenant, per-endpoint limit. Exceeding it returns `429 Too Many Requests` with a `Retry-After` header.

Figure 1.1 -- A single request/response round trip, tenant-filtered at the database layer.

2. Authentication

CONCEPT · JWT (JSON Web Token)

WHAT IT IS

A JWT is a small, digitally signed piece of text that proves who you are and what you're allowed to do, without the server needing to look you up in a database on every single request.

REAL-WORLD ANALOGY

It's like a wristband at a festival. Security checks your ID once at the gate and puts a signed wristband on you. For the rest of the day, staff just glance at the wristband -- they don't re-check your ID at every stage door, but the wristband can't be faked or altered without it being obvious.

WHY STAYKARO USES IT

The voice gateway, both dashboards, and background workers all need to prove their identity and tenant on every request. Looking up a session in the database on every single call would add latency exactly where the Architecture Spec's §20 latency budget can't afford it.

TECHNICAL IMPLEMENTATION

Issued by `POST /api/v1/auth/login`, signed with a server-held secret, containing `user_id, tenant_id, role, exp` (expiry). Verified on every request by `packages/auth` before any handler code runs.

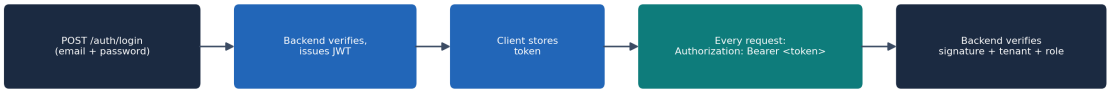

Figure 2.1 -- Login issues a token once; every subsequent request carries and re-verifies it.

SECURITY NOTE

A JWT proves identity and role -- it does not, by itself, prove tenant scoping is safe. Every handler still resolves tenant context and applies RLS per Architecture Spec §10. Never trust a tenant_id that arrives in a request body or query string over the one embedded in the verified token.

3. Authentication Endpoints

POST `/api/v1/auth/login`

Purpose: Exchange email + password for a JWT access token and refresh token.

Auth: None -- this endpoint issues the credential

Used by: Login screens, both dashboards

Owned by: Nishkala (packages/auth)

Request body

```
{
  "email": "ops@staykaro.com",
  "password": "••••••••"
}
```

Response body (success)

```
{
  "access_token": "eyJhbGciOi...",
  "refresh_token": "eyJhbGciOi...",
  "expires_in": 3600,
  "role": "client_admin",
  "tenant_id": "tenant_123"
}
```

Status codes

Code	Meaning
200	Login successful
401	Invalid email or password
429	Too many failed attempts -- rate limited

Validation rules

- email is a valid email format

- password is non-empty
- Failed attempts are rate-limited per §1.3 to slow credential-stuffing attempts

Security notes

- Password is verified against a bcrypt hash, never stored or logged in plain text
- Response never indicates whether the email or the password was wrong, only that the pair was invalid

POST `/api/v1/auth/refresh`

Purpose: Exchange a valid refresh token for a new access token, without re-entering credentials.

Auth: Refresh token (not access token) **Used by:** Both dashboards (silent session renewal) **Owned by:** Nishkala (packages/auth)

Request body

```
{
  "refresh_token": "eyJhbGciOi..."
}
```

Response body (success)

```
{
  "access_token": "eyJhbGciOi...",
  "expires_in": 3600
}
```

Status codes

Code	Meaning
200	New access token issued
401	Refresh token invalid, expired, or revoked

Validation rules

- refresh_token must be a well-formed, unexpired JWT of type refresh

Security notes

- Refresh tokens are revocable server-side (e.g. on logout or password change) -- verification checks a revocation list, not just the signature

POST `/api/v1/auth/logout`

Purpose: Revoke the current refresh token.

Auth: Access token **Used by:** Both dashboards **Owned by:** Nishkala (packages/auth)

Response body (success)

```
{
  "status": "logged_out"
}
```

Status codes

Code	Meaning
200	Logged out
401	Already invalid/expired token

Validation rules

- N/A -- no body

Security notes

- Revokes only the current session's refresh token, not every session for that user, unless a 'logout everywhere' flag is added later

4. Agent Configuration Endpoints

GET `/api/v1/agents/{agent_id}`

Purpose: Fetch an AI agent's current configuration (prompt, voice, languages, phone number).

Auth: Access token, role: client_admin or client_user for own tenant; any Staykaro admin role

Used by: Client Dashboard agent-config screen (NH-09)

Owned by: Nishkala (services/api)

Response body (success)

```
{
  "agent_id": "sales_agent_01",
  "tenant_id": "tenant_123",
  "prompt": "You are ABC Motors' assistant...",
  "voice": "cartesia:hi-warm-1",
  "languages": ["en-IN", "hi-IN", "te-IN"],
  "phone_number": "+91XXXXXXXXXX"
}
```

Status codes

Code	Meaning
200	Returned
403	Requesting tenant does not own this agent
404	No such agent

Validation rules

- agent_id is a valid identifier format

Security notes

- tenant_id on the returned agent is compared against the token's tenant_id before any data leaves the handler -- a 403, not a 404, is returned on mismatch, to avoid confirming whether the ID merely doesn't exist vs. belongs to someone else being ambiguous in a way that leaks less information either way

PATCH `/api/v1/agents/{agent_id}`

Purpose: Update an agent's prompt, voice, or language configuration.

Auth: Access token, role: client_admin for own tenant, or Staykaro admin

Used by: Client Dashboard agent-config screen (NH-09)

Owned by: Nishkala (services/api)

Request body

```
{
  "prompt": "Updated system prompt text...",
  "voice": "cartesia:hi-warm-2"
}
```

Response body (success)

```
{
  "agent_id": "sales_agent_01",
  "updated_at": "2026-07-26T10:15:00Z"
}
```

Status codes

Code	Meaning
200	Updated
400	Invalid field value
403	Not this tenant's agent
404	No such agent

Validation rules

- prompt, if present, is non-empty and under the configured max length
- voice, if present, must be a known provider:voice_id pair

Security notes

- Change is written to audit_logs (Architecture Spec §27) with before/after values
- Takes effect on the agent's next call, not mid-call

5. Knowledge Base Endpoints

■ QUICK REMINDER — Embeddings / Vector Search

A number-based fingerprint of meaning, used so the system can find knowledge chunks related to a caller's question even if the wording doesn't match exactly. *(Full explanation: Developer Handbook.)* See §14-15 of the Database Design document for the full explanation and how pgvector stores and searches these.

POST `/api/v1/knowledge/documents`

Purpose: Upload a new knowledge document (PDF, DOCX, or TXT) for the agent's RAG knowledge base.

Auth: Access token, role: client_admin for own tenant

Used by: Client Dashboard knowledge-base screen (NH-10)

Owned by: Nishkala (services/worker/ingestion)

Request body

```
multipart/form-data: file=<binary>, filename=<string>
```

Response body (success)

```
{
  "document_id": "doc_88a2",
  "filename": "pricing.pdf",
  "status": "processing"
}
```

Status codes

Code	Meaning
202	Accepted -- processing begins asynchronously
400	Unsupported file type or file too large
403	Not this tenant

Validation rules

- File type must be PDF, DOCX, or TXT
- File size under the configured limit (prevents a single upload from monopolizing worker capacity)

Security notes

- File is scanned before extraction (Architecture Spec §14 'security validation')
- document_id is tagged with tenant_id at creation -- this tag, not the uploading user's session alone, is what every later retrieval filters on

GET `/api/v1/knowledge/documents`

Purpose: List all knowledge documents for the caller's tenant, with processing status.

Auth: Access token

Used by: Client Dashboard knowledge-base screen

Owned by: Nishkala

Response body (success)

```
{
  "documents": [
    {"document_id": "doc_88a2", "filename": "pricing.pdf", "status": "ready", "chunk_cou
nt": 42}
  ],
  "pagination": {"total": 6, "page": 1, "page_size": 25, "has_next": false}
}
```

Status codes

Code	Meaning
200	Returned

Validation rules

- page / page_size follow §1.2

Security notes

- Query is filtered by tenant_id server-side -- never accepts a tenant_id parameter from the client

DELETE `/api/v1/knowledge/documents/{document_id}`

Purpose: Remove a document and its derived chunks/embeddings from the knowledge base.

Auth: Access token, role: client_admin for own tenant

Used by: Client Dashboard knowledge-base screen

Owned by: Nishkala

Response body (success)

```
{
  "document_id": "doc_88a2",
  "status": "deleted"
}
```

Status codes

Code	Meaning
200	Deleted
403	Not this tenant’s document
404	No such document

Validation rules

- N/A

Security notes

- Deletes all derived knowledge_chunks rows for this document, not just the document row -- a dangling embedding pointing at deleted source text would let the agent cite content that no longer officially exists

6. Call Endpoints

GET `/api/v1/calls`

Purpose: List calls for the caller's tenant, most recent first.

Auth: Access token

Used by: Client Dashboard call history (NH-11), Admin Dashboard (NH-06/07)

Owned by: Nishkala

Response body (success)

```
{
  "calls": [
    {"call_id": "call_928abc", "started_at": "2026-07-26T09:02:00Z",
     "duration_seconds": 184, "language": "te-IN", "status": "completed"}
  ],
  "pagination": {"total": 340, "page": 1, "page_size": 25, "has_next": true}
}
```

Status codes

Code	Meaning
200	Returned

Validation rules

- Optional filters: status, language, date_from, date_to

Security notes

- Admin-role requests may pass a tenant_id filter explicitly; client-role requests always resolve to their own tenant regardless of any parameter supplied

GET `/api/v1/calls/{call_id}/transcript`

Purpose: Fetch the full, turn-by-turn transcript for a completed call.

Auth: Access token

Used by: Client Dashboard transcript view, Admin live-call/history view

Owned by: Nishkala

Response body (success)

```
{
  "call_id": "call_928abc",
  "turns": [
    {"turn_id": "turn_001", "speaker": "caller", "language_code": "te-IN",
     "text": "...", "started_at": "2026-07-26T09:02:04Z"},
    {"turn_id": "turn_002", "speaker": "agent", "language_code": "te-IN", "text": "..."}
  ]
}
```

Status codes

Code	Meaning
200	Returned
403	Not this tenant's call
404	Transcript not yet finalized or call not found

Validation rules

- N/A

Security notes

- Transcript text is caller-generated content -- access is logged to audit_logs when accessed by a Staykaro admin role (not by the tenant viewing their own data), per PII-handling policy in Architecture Spec §27

7. Usage & Billing Endpoints

GET `/api/v1/usage/summary`

Purpose: Return usage totals for the caller's tenant for a given period, broken down by resource type.

Auth: Access token

Used by: Client Dashboard usage screen, Admin usage/cost screen (NH-08)

Owned by: Nishkala (packages/billing)

Response body (success)

```
{
  "period": "2026-07",
  "telephony_seconds": 48210,
  "stt_seconds": 47990,
  "llm_input_tokens": 812000,
  "llm_output_tokens": 265000,
  "tts_characters": 940000,
  "call_count": 312
}
```

Status codes

Code	Meaning
200	Returned
400	Invalid period format

Validation rules

- period must be YYYY-MM

Security notes

- Provider-cost and margin fields are included only for Staykaro admin roles, never for client roles -- a client should see their own usage, not Staykaro's cost structure or margin on serving them

GET /api/v1/billing/invoices

Purpose: List invoices for the caller's tenant.

Auth: Access token

Used by: Client Dashboard billing screen (NH-12)

Owned by: Nishkala

Response body (success)

```
{
  "invoices": [
    {"invoice_id": "inv_0091", "period": "2026-06", "amount_inr": 24999,
     "status": "paid", "issued_at": "2026-07-01T00:00:00Z"}
  ]
}
```

Status codes

Code	Meaning
200	Returned

Validation rules

- N/A

Security notes

- Amounts reflect what Razorpay has actually recorded, not a locally recalculated figure -- avoids the dashboard ever showing a number that disagrees with the actual payment processor

POST /api/v1/webhooks/razorpay

Purpose: Receive payment/subscription lifecycle events from Razorpay.

Auth: Razorpay webhook signature (not a user JWT)

Used by: Razorpay (server-to-server, no UI)

Owned by: Nishkala (packages/billing)

Request body

```
Razorpay-defined event payload -- see Razorpay's own webhook documentation for the exact
 shape per event type
```

Response body (success)

```
{
  "received": true
}
```

Status codes

Code	Meaning
200	Processed (or already processed -- see idempotency note)
400	Signature verification failed

Validation rules

- Signature header is verified against the Razorpay webhook secret before the body is trusted at all

Security notes

- Idempotent by event ID (Architecture Spec §22) -- a duplicate delivery of the same event returns 200 without reapplying the billing-state change a second time
- Always returns 200 quickly once verified, even if downstream processing is queued, so Razorpay does not retry unnecessarily

8. Admin Endpoints

Everything in this section requires a Staykaro internal role (Super Admin, Operations, or Support per Architecture Spec §26) -- never a client role, regardless of what tenant_id is requested.

GET `/api/v1/admin/clients`

Purpose: List all client tenants on the platform, with plan and status.

Auth: Access token, Staykaro Operations role or above

Used by: Admin Dashboard client list (NH-06)

Owned by: Nishkala

Response body (success)

```
{
  "clients": [
    {"tenant_id": "tenant_123", "name": "ABC Motors", "plan": "growth",
     "status": "active", "phone_numbers": 1}
  ],
  "pagination": {"total": 14, "page": 1, "page_size": 25, "has_next": false}
}
```

Status codes

Code	Meaning
200	Returned
403	Caller does not hold an internal Staykaro role

Validation rules

- N/A

Security notes

- This is the one list endpoint in the whole API that legitimately spans tenants -- it is the highest-value target for a broken-access-control bug, and is covered explicitly by the tenant-isolation suite (Execution Plan NK-08) to confirm client-role tokens are rejected here, not merely unlinked in the UI

PATCH `/api/v1/admin/clients/{tenant_id}`

Purpose: Update a client's plan or account status (e.g. suspend, reactivate).

Auth: Access token, Staykaro Super Admin or Operations role

Used by: Admin Dashboard client details (NH-06)

Owned by: Nishkala

Request body

```
{
  "plan": "scale",
  "status": "active"
}
```

Response body (success)

```
{
  "tenant_id": "tenant_123",
  "updated_at": "2026-07-26T10:20:00Z"
}
```

Status codes

Code	Meaning
200	Updated
403	Insufficient role
404	No such tenant

Validation rules

- plan must be one of the configured plan identifiers
- status transitions follow an allowed state machine (e.g. active → suspended → active; not active → deleted directly)

Security notes

- Written to audit_logs with the acting admin's user_id -- account status changes are exactly the kind of action a support ticket dispute needs a paper trail for

9. System Health Endpoints

GET `/api/v1/health`

Purpose: Liveness check -- is the API process running at all.

Auth: None

Used by: Docker health check, uptime monitoring (Nihal, NH-18)

Owned by: Nishkala (NK-04)

Response body (success)

```
{
  "status": "ok"
}
```

Status codes

Code	Meaning
200	Process is alive

Validation rules

- N/A

Security notes

- Deliberately does not check downstream dependencies -- that's /health/ready below. A liveness check that depends on the database means a database blip restarts a perfectly healthy API process, which is the opposite of what you want.

GET `/api/v1/health/ready`

Purpose: Readiness check -- can the API actually serve requests right now (DB, Redis, and critical providers reachable).

Auth: None

Used by: Deployment draining logic (NH-17), load balancer / orchestration health checks

Owned by: Nishkala (NK-04)

Response body (success)

```
{
  "status": "ready",
  "checks": {"postgresql": "ok", "redis": "ok"}
}
```

Status codes

Code	Meaning
200	Ready to serve traffic
503	One or more dependencies unavailable -- do not route traffic here

Validation rules

- N/A

Security notes

- This is the endpoint the deployment-draining sequence (Architecture Spec §30) uses to decide whether an instance is safe to send new calls to

10. Internal Endpoints (Voice Gateway → API)

These are not called by any dashboard. They're the contract between Shivashree's voice gateway and Nishkala's backend -- listed in the API Contract Ownership Table, Execution Plan §J -- and are reachable only from inside the private Docker network, never exposed through the public Nginx routes.

POST `/internal/v1/calls`

Purpose: Create a call record the moment a call connects, before any turns have happened.

Auth: Internal service token (voice-gateway ↔ api), not a user JWT **Used by:** Voice Gateway (SH-03) **Owned by:** Nishkala (NK-12)

Request body

```
{
  "call_id": "call_928abc",
  "tenant_id": "tenant_123",
  "agent_id": "sales_agent_01",
  "started_at": "2026-07-26T09:02:00Z"
}
```

Response body (success)

```
{
  "call_id": "call_928abc",
  "status": "created"
}
```

Status codes

Code	Meaning
201	Created
409	call_id already exists (duplicate connect event)

Validation rules

- call_id is generated by the voice gateway and must be globally unique

Security notes

- Rejects a call_id that doesn't map to a real tenant_id/agent_id pair -- prevents a malformed or spoofed internal call from creating an orphaned record

POST `/internal/v1/calls/{call_id}/turns`

Purpose: Append one conversational turn (caller or agent) to a call's history.

Auth: Internal service token **Used by:** Voice Gateway (SH-11) **Owned by:** Nishkala (NK-12)

Request body

```
{
  "turn_id": "turn_004",
  "speaker": "caller",
  "language_code": "te-IN",
  "text": "...",
  "started_at": "2026-07-26T09:02:41Z"
}
```

Response body (success)

```
{
  "turn_id": "turn_004",
  "status": "recorded"
}
```

Status codes

Code	Meaning
201	Recorded
404	call_id does not exist or already finalized

Validation rules

- turn_id is unique within the call and strictly increasing
- language_code is a recognized ISO code

Security notes

- This is the write path that ultimately populates the transcript endpoint in §6 -- any bug here is a data-integrity bug, not just a display bug, per Architecture Spec §11's call_turns schema

POST `/internal/v1/calls/{call_id}/finalize`

Purpose: Mark a call complete, triggering usage calculation and the post-call automation event.

Auth: Internal service token

Used by: Voice Gateway (SH-16, on telephony disconnect)

Owned by: Nishkala (NK-12, NK-13)

Request body

```
{
  "ended_at": "2026-07-26T09:05:24Z",
  "end_reason": "caller_hangup"
}
```

Response body (success)

```
{
  "call_id": "call_928abc",
  "status": "finalized",
  "duration_seconds": 204
}
```

Status codes

Code	Meaning
200	Finalized
404	call_id not found
409	Already finalized (duplicate disconnect event)

Validation rules

- end_reason is one of a fixed set: caller_hangup, agent_ended, provider_failure, timeout

Security notes

- Finalizing is idempotent -- a duplicate finalize call (e.g. from a retried webhook) does not double-count usage or fire the post-call automation event twice

11. Standard Error Response Format

CONCEPT · Why every error looks the same, no matter which endpoint failed

WHAT IT IS

Instead of each endpoint inventing its own error shape, every failing request across the entire API returns the same JSON structure, with the details specific to that failure filled in.

REAL-WORLD ANALOGY

It's like every department in a company using the same incident-report form. You don't need to learn a new form to understand what went wrong in a department you've never dealt with before.

WHY STAYKARO USES IT

Nihal's dashboards can show a sensible error message for any failure from any endpoint using one shared piece of frontend code, instead of custom error-handling per screen.

TECHNICAL IMPLEMENTATION

Every error response is shaped by the `StaykaroError` hierarchy from the Developer Handbook §7.

```
{
  "error": {
    "code": "cross_tenant_access_denied",
    "message": "You do not have access to this resource.",
    "request_id": "req_9f2ac31b"
  }
}
```

Figure 11.1 -- Standard error shape. request_id lets support look up the exact log entry in seconds.

HTTP status	Used when
400	Request failed validation (bad input shape or values)
401	Missing, invalid, or expired authentication
403	Authenticated, but not permitted -- including any cross-tenant attempt
404	Resource does not exist (or, for cross-tenant reads, is deliberately indistinguishable from not existing)
409	Conflict with current state (duplicate, already-finalized, etc.)
429	Rate limited
500	Unhandled server error -- always logged with a request_id, never shown with internal detail to the client
503	A dependency is unavailable -- see /health/ready, §9